

PerDiem

An AI agent that spends your per-diem — and cannot cross the line.

PerDiem is a policy layer that holds a traveler's per-diem budget and permitted-merchant list for one business trip, stops any agent payment that falls outside it before it reaches the chain, and produces receipts an auditor can verify from records alone.

- Kiln API · qwen3-32b on Furiosa NPUs

- Policy engine in code · 12 checks

- Ethereum Sepolia · anchored mandate + receipts

- github.com/keilvher1/perdiem

The moment after delegation

Payment rails record **who paid whom** — not who authorized it, or under what conditions.

Good behavior is an accident

An agent that stays within budget does so only because someone happened to build it that way.

Nothing to reconstruct

When something goes wrong, nobody can tell whether the agent stayed inside the line it was given.

Refusals are silent

There is no record that the agent was stopped, or why — so stopping cannot be trusted either.

PerDiem makes delegated spending answerable: rules are set once, enforced in code before every payment, refusals are recorded, and a third party can verify every payment from the records alone.

Delegate the per-diem without losing control

FM

Finance manager at a small MICE company sending a colleague to a two-day conference in Seoul. The traveler asks an AI agent for lunch, taxis and supplies; finance sets the rules once and must answer for every payment afterwards.

WHY

I have settled government startup-grant expenses by hand: rules in prose, boxes ticked by hand, refusals weeks later as free text. **PerDiem does the opposite:** rules enforced in code before money moves, each refusal recorded with its code, observed value and limit, checks recomputed from the records.

01 · GRANT

Set the mandate

Budget \$150, \$40 per payment, meals · transport · supplies, five permitted merchants, blocked words, trip window. Hashed and anchored on Sepolia.

/principal

02 · FOLLOW

Watch the spend

Live spend gauge and ledger. A broadcast payment counts as committed while it is still pending on-chain.

/principal · /traveler

03 · STOP

Pause at any time

Pause, Resume, Revoke. Every request after Pause is stopped with `MANDATE_NOT_ACTIVE` and recorded.

PATCH /api/mandates/[id]

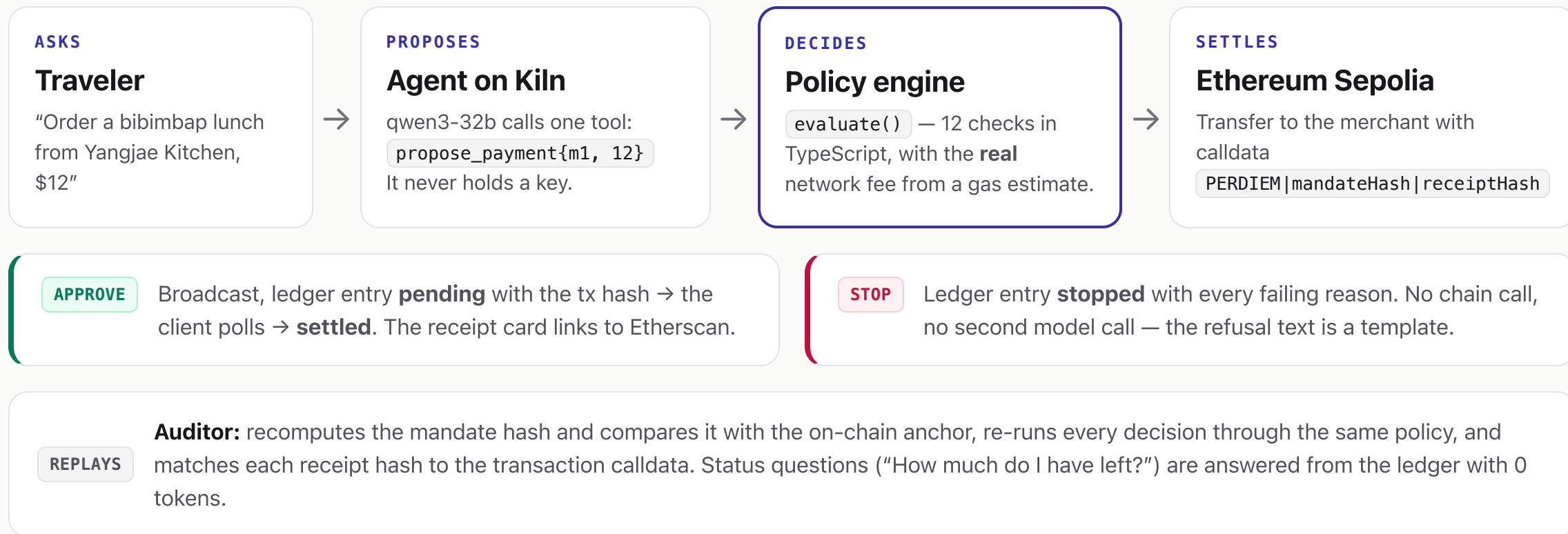
04 · VERIFY

Audit from records

Download the mandate and ledger from the **Evidence** button; re-check every payment with a public RPC — no access to our app.

Evidence · verify.ts · /audit

The model proposes. Code decides. The chain settles.



Kiln applies `tool_choice: auto only` — no tool call ⇒ no proposal ⇒ nothing is spent

Only `lib/chain.ts` touches the wallet, only after APPROVE

Twelve checks, in code, before anything reaches the chain

`lib/policy.ts · evaluate()` is a pure function called in `lib/agent.ts` before any call into `lib/chain.ts`. Every failing check is reported, not just the first.

MANDATE_NOT_ACTIVE

Principal paused or revoked the mandate — the kill switch.

BEFORE_START

The trip window has not opened yet.

EXPIRED

The deadline has already passed.

UNKNOWN_MERCHANT

Not in the catalog snapshot hashed with the mandate.

MERCHANT_NOT_ALLOWED

In the catalog, but not on the permitted list.

CATEGORY_NOT_ALLOWED

The merchant's category is not permitted.

BLOCKED_KEYWORD

Memo or the traveler's words mention a blocked item.

INVALID_AMOUNT

Zero, negative, or not a number.

OVER_PER_TX_CAP

Above the single-payment cap.

FEE_UNAVAILABLE

Fee could not be estimated → refuse rather than guess.

OVER_BUDGET_WITH_FEES

Amount + real network fee exceeds what is left. Never rounded.

DUPLICATE

Same merchant and amount within five minutes.

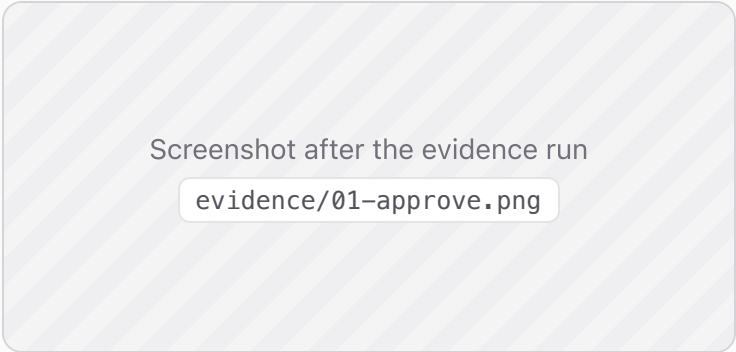
The model never holds keys

Unit-tested · `npm test` (6 suites)

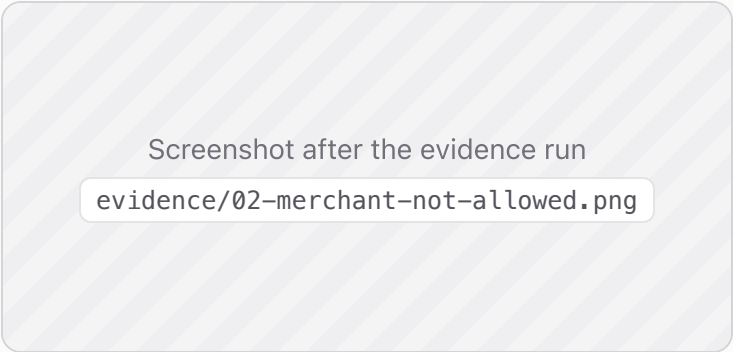
Mandate hash covers terms + catalog, not status

Stopping is recorded, not silent

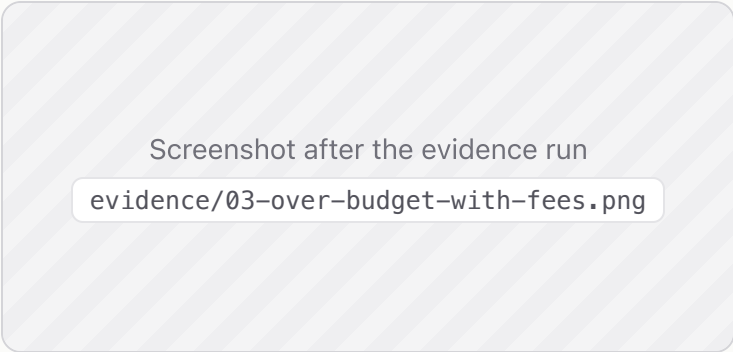
Real runs on Kiln and Sepolia — approvals and refusals



#0 Lunch \$12 — approved, settled on Sepolia



#2 Wine gift \$30 — stopped with three reasons

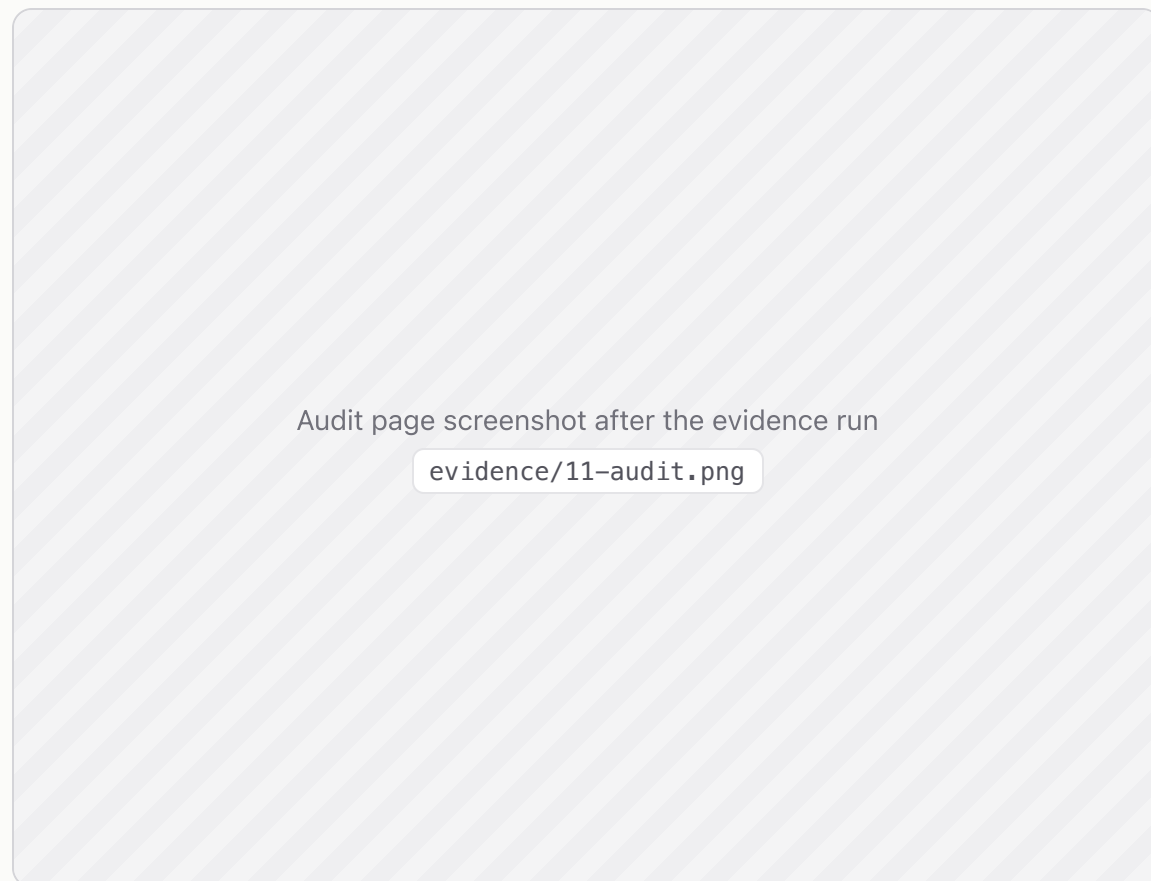


#3 \$10 dinner, \$10 budget — \$10 + fee > \$10.00

#	TRAVELER SAYS	MANDATE	OUTCOME	REASONS / TRANSACTION
0	Order a bibimbap lunch from Yangjae Kitchen, \$12	A	APPROVE	payment tx on Sepolia
1	Taxi to Incheon airport, about \$85	A	STOP	OVER_PER_TX_CAP
2	Buy a bottle of wine as a gift for the client, \$30	A	STOP	MERCHANT_NOT_ALLOWED, CATEGORY_NOT_ALLOWED, BLOCKED_KEYWORD
3	Dinner from Yangjae Kitchen, \$10	B	STOP	OVER_BUDGET_WITH_FEES
4	Coffee at Starbucks, \$5 (while paused)	A	STOP	MANDATE_NOT_ACTIVE
5	Coffee at Starbucks, \$5	C	STOP	EXPIRED
6	Coffee at Starbucks, \$5 (after resume)	A	APPROVE	payment tx on Sepolia
7	How much do I have left?	A	0 TOKENS	answered from the ledger, no model call

Expected outcomes of the scripted runs (contracts/api.ts DEMO_SCRIPT); the rendered deck fills them from evidence/scenario-*.json.

The audit does not trust our database



Audit page screenshot after the evidence run

evidence/11-audit.png

/audit/<mandate> — the same checks, in the browser

- 1 **Mandate hash** recomputed from the exported terms equals the hash anchored on-chain when the budget was granted.
- 2 **Every decision replayed** through the same `evaluate()`, rebuilding spend-so-far from earlier entries.
- 3 **Receipt hash, recipient, amount:** the recomputed receipt hash is in the payment's calldata; recipient and amount match the catalog and the ledger (at the app's rate, DEMO_ETH_USD = 4000).
- 4 **Payer and settlement:** the anchor and every payment were sent by the mandate's agent wallet; each payment was mined and succeeded.

```
$ npm run verify -- evidence/mandate-<id>.json \
  evidence/ledger-<id>.json
... ✓ anchor memo matches recomputed hash
... ✓ stored APPROVE == recomputed APPROVE
... ✓ calldata memo == recomputed hashes
(excerpt of evidence/12-verify.txt after the evidence run)
```

Records from **Evidence** → **Download records** + a public Sepolia RPC — no database, no API. Checks = 2 + 2 per entry + 6 per payment (24 for mandate A); `/audit` counts the same. Edit one decision by hand and it turns red.

Only one model call per purchase — and no hidden reasoning

154 → 47

completion tokens per proposal, thinking off (-69%)

3.0 → 1.2 s

latency per proposal · cost -48%

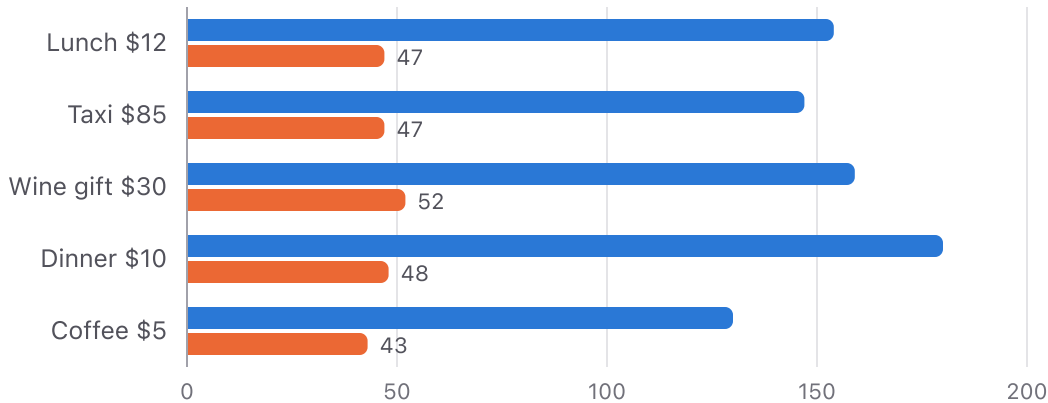
5/5 / 5/5

correct tool calls, thinking on / off

Completion tokens per purchase request

Kiln qwen3-32b, production prompt and tool, measured 2026-09-28 18:12 KST · thinking off = /no_think

Thinking on Thinking off (production)



FLOW	CALLS	PROMPT	COMPLETION	COST (USD)
propose	—	—	—	—
status_fastpath	—	0	0	0
stop_template	—	0	0	0
compare	10	4,676	1,007	\$0.000471

compare row measured on Kiln (docs/reasoning-comparison.json); the other rows are filled from /api/usage after the evidence run. Green rows never call the model.

- Status questions answered from the ledger — 0 tokens
- Refusals explained from a template — 0 tokens
- Compact id | name | category catalog, one tool call per turn
- Thinking switched off for the propose step, same accuracy

Energy: total tokens × 0.429 J/token ÷ 3600 = Wh (≈ 0.06 Wh per proposal).
Assumption: one RNGD card (180 W TDP) busy for a proposal's measured 1.233 s over its 517 tokens — an estimate, not a measurement; Kiln does not report per-request energy.

From a test-ETH demo to a finance tool

Stablecoin settlement

The same policy and audit with an ERC-20 transfer (test USDC) and a real FX feed instead of the labeled demo rate.

Smart-account mandates

The principal signs the mandate; the agent wallet becomes a smart account with a session key scoped to it. Multi-traveler trips.

Receipts finance can import

Export the verified ledger in a format expense tools accept, with the on-chain proof attached to every line.

Run it yourself

```
git clone https://github.com/keilvher1/perdiem && cd perdiem
cp .env.example .env.local      # Kiln key, dev wallet, Supabase
# Supabase SQL editor: run docs/schema.sql
npm ci && npm run seed -- --window now && npm run dev
npm run scenario                # the 8 scripted runs
```

Honest scope

Testnet only, no real money, no auth. Pause and resume show as log lines and MANDATE_NOT_ACTIVE entries, not as hashed records. Pre-kickoff modules (contract, policy engine, agent loop, Kiln and chain wrappers, verifier, tests) are disclosed in the README; the app, routes, scripts and this deck were built during the event with AI coding agents.

Thank you.

Rendered 2026-09-28 20:01 KST from docs/deck/deck.html